

Compute, and what it costs

All-purpose against jobs compute, the three SQL warehouse types, access modes, and the levers that actually move the bill.

Databricks sends you **two bills**. One from Databricks for DBUs, one from your cloud provider for the machines. The compute type you pick changes the first one by two to three times.

The compute types

TYPE	WHAT IT IS FOR	LIFECYCLE
All-purpose	Interactive notebooks and development	Runs until stopped or idle timeout
Jobs compute	Scheduled and automated workloads	Starts with the job, terminates when it ends
SQL warehouse	SQL queries, dashboards and BI tools	Auto-stops after an idle period
Serverless	Any of the above, managed by Databricks	Starts in seconds, no idle cost
Instance pool	Warm VMs that cut start-up time	Cloud cost while idle, no DBUs

ON THE EXAM

Jobs compute ends when the work ends. All-purpose ends when someone remembers.

All-purpose or jobs compute?

ALL - PURPOSE

Built for people

- Attach a notebook and run cells interactively
- Shared across a team, so jobs compete for resources
- Auto-termination is tied to inactivity, not to work finishing
- The more expensive DBU rate

JOBS COMPUTE

Built for schedules

- Created for the run, destroyed after it
- Isolated, so no competition from other users
- Terminates when the job completes, always
- Roughly two to three times cheaper per DBU

THE TIEBREAKER

Databricks explicitly recommends against running jobs on all-purpose compute. It is the most common avoidable cost on the platform.

Why Databricks recommends against jobs on all-purpose

Four reasons, straight from the documentation

- ◆ **Different billing rate.** All-purpose is billed at a higher DBU rate than jobs compute for identical work.
- ◆ **Termination is tied to inactivity, not completion.** The cluster idles on after your job has finished.
- ◆ **Shared across teams.** Scheduled jobs get increased latency from competing for resources.
- ◆ **Tuning advice does not transfer.** Settings good for ad-hoc interactive work are wrong for batch.
- ◆ **The one accepted exception:** iteratively developing or testing a new job, where jobs-compute start-up makes each cycle tedious.

ON THE EXAM

Development is a legitimate reason. 'The cluster was already running' is not.

Serverless or classic compute?

CLASSIC

You own the cluster

- Runs in your cloud account and network
- Minutes to start, unless you use a pool
- You choose node types, versions and libraries
- Idle time bills until termination fires

SERVERLESS

Databricks owns the cluster

- Runs in the Databricks account, same region
- Starts in seconds; nothing to provision
- Fewer knobs, and fewer ways to misconfigure
- No idle cost, because there is no idle

THE TIEBREAKER

Bursty short work favours serverless. Long steady runs, or a need for specific node types, favour classic.

The three SQL warehouse types

	CLASSIC	PRO	SERVERLESS
Photon	Yes	Yes	Yes
Predictive IO	No	Yes	Yes
Intelligent Workload Management	No	No	Yes
Compute runs in	Your account	Your account	Databricks account
Typical start-up	~4 minutes	~4 minutes	Seconds

ON THE EXAM

Serverless is the only type whose compute layer sits in the Databricks account, and the only one that starts in seconds.

Sizing and stopping a warehouse

- ◆ **Size is about one query's speed.** A bigger warehouse makes an individual query faster by giving it more workers.
- ◆ **Scaling is about many queries at once.** More clusters handle more concurrent users, not faster single queries.
- ◆ **Sizing up to fix a queue is the classic mistake.** Queuing is a concurrency problem, so scale out instead.
- ◆ **Auto-stop is the idle-cost lever.** Serverless defaults to stopping after ten minutes idle and can be set lower.

ON THE EXAM

A busy dashboard keeps a warehouse warm on its own. Aggressive auto-stop costs nothing when traffic is steady.

Access modes were renamed. **Single user** is now **dedicated**. **Shared** is now **standard**. Older study material uses the old names throughout.

The three access modes

MODE	FORMERLY	UNITY CATALOG	USE IT WHEN
Standard	Shared	Yes, fully enforced	Almost always. The recommended default.
Dedicated	Single user or assigned	Yes	Standard cannot do what you need
No isolation shared	Unchanged	No	Legacy hive_metastore work only

ON THE EXAM

Only standard and dedicated can access Unity Catalog data. If a question needs UC governance, no isolation shared is always wrong.

When dedicated is actually required

- ◆ **Databricks Runtime ML**, including GPU workloads. Not supported on standard access mode.
- ◆ **The RDD API**, rather than DataFrames or SQL.
- ◆ **R workloads.**
- ◆ **Dedicated can be assigned to a group**, not only a person, from Databricks Runtime 15.4 upwards.

ON THE EXAM

On a group-dedicated resource, a user's permissions down-scope to the group's. The group must own the privileges, not the individual.

Photon

- ◆ **A vectorised query engine written in C++**, replacing parts of the JVM execution path. Your code does not change.
- ◆ **Helps most on** large scans, aggregations, joins and writes to Delta and Parquet.
- ◆ **Helps least on** queries already finishing in a couple of seconds, and on non-Spark Python running in a UDF.
- ◆ **Costs a higher DBU rate**, and can still be cheaper overall, because billing is by time and the job finishes sooner.

ON THE EXAM

Photon is enabled by default on all SQL warehouse types. On clusters it is a checkbox.

What a cluster is made of

01

Driver

Holds the notebook state, plans work, collects results.

02

Executors

Worker nodes that do the actual processing.

03

Cores

Each executor runs tasks in parallel, one per core.

04

Tasks

A partition of data processed by one core.

WHY THIS ASYMMETRY MATTERS

One driver, many executors. Lose an executor and Spark re-runs its tasks. Lose the driver and the whole cluster dies.

Fixed size or autoscaling?

FIXED SIZE

Predictable

- Same number of workers throughout
- No time lost adding or removing nodes
- Predictable cost per run
- Good when the workload is steady and known

AUTOSCALING

Elastic

- Workers added when tasks queue, removed when idle
- You set a minimum and a maximum
- Good for variable or unpredictable load
- Scaling itself takes time, so it suits longer runs

THE TRAP

Autoscaling is not free. Adding nodes takes minutes, so on a short job the scale-up finishes about when the job does.

Three settings that cost or save real money

- ◆ **Auto-termination.** Shuts an idle all-purpose cluster down. Defaults are generous, commonly 120 minutes.
- ◆ **Spot or preemptible instances.** Spare cloud capacity at a large discount, reclaimable at any moment.
- ◆ **Instance pools.** Warm VMs that clusters borrow, cutting start-up from minutes to seconds.
- ◆ **Never put the driver on spot.** Losing an executor costs a retry. Losing the driver kills the job.

ON THE EXAM

Pools bill you for idle VMs at the cloud provider's rate but consume no DBUs while idle. They trade a small standing cost for fast starts.

Databricks Runtime

- ◆ **A versioned image** containing Spark, Delta Lake, Python, Java, Scala, R and preinstalled libraries.
- ◆ **LTS versions are long-term support.** Databricks recommends the latest LTS for production.
- ◆ **ML variants** add machine learning libraries and GPU support, and require dedicated access mode.
- ◆ **Features have version floors.** Group-dedicated compute needs 15.4 or above, for example.

ON THE EXAM

When a question says a feature 'is not available', check the runtime version before concluding the approach is wrong.

Cluster policies

- ◆ **Admin-defined rules** constraining what compute a user may create.
- ◆ **Can force an access mode**, so nobody creates compute that bypasses Unity Catalog.
- ◆ **Can cap node counts and types**, and require auto-termination below a limit.
- ◆ **Can enforce tags**, which is how cost gets attributed to a team or project.

ON THE EXAM

Policies answer questions about preventing a mistake, rather than detecting it afterwards.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
A scheduled pipeline on a shared interactive cluster	Move it to jobs compute
First dashboard query each morning is slow	Classic or pro warehouse cold start, ~4 minutes
Queries queue when many analysts arrive	Scale out to more clusters, not up a size
Needs Databricks Runtime ML, GPU, RDD or R	Dedicated access mode
Governed access to Unity Catalog data	Standard or dedicated, never no isolation shared
Cutting cost on a critical job	Spot for workers, on-demand for the driver

ON THE EXAM

Six lines. Each one is a whole family of exam questions compressed into a sentence.

A team runs five nightly ETL pipelines. All five are scheduled against a shared all-purpose cluster that the team also uses for development during the day. The cluster has auto-termination set to 120 minutes.

The pipelines take about 95 minutes of processing in total, spread across the night. Finance has asked the team to reduce the cost of this workload.

Which change reduces cost the most?

- ☐ A Reduce auto-termination on the all-purpose cluster from 120 minutes to 30 minutes
- ☐ B Enable autoscaling on the all-purpose cluster so it shrinks between pipeline runs
- ☐ C Schedule the five pipelines against jobs compute instead
- ☐ D Enable Photon on the all-purpose cluster so the pipelines finish sooner

◆ PAUSE AND DECIDE

Which change reduces cost the most?

- A

 Cut auto-termination to 30 minutes
Genuinely helps, and it is the trap. It shortens idle windows but leaves every minute of actual work billing at the all-purpose rate.
- B

 Enable autoscaling between runs
Shrinks the cluster during gaps, but the driver keeps running and the rate is unchanged. A partial fix to half the problem.
- C

 Move the pipelines to jobs compute
Correct. It fixes both halves at once: the DBU rate drops two to three times, and each cluster terminates when its run ends rather than idling.
- D

 Enable Photon
May shorten runtime, but Photon costs a higher DBU rate and the workload still sits on the wrong compute type. Optimising the wrong thing.

THE SIGNAL

Two costs are in play: the rate and the idle time. A, B and D each address one. Only C addresses both.

Analysts report that the first dashboard load each morning takes around four minutes, after which everything is fast for the rest of the day. The dashboard runs on a classic SQL warehouse with auto-stop set to 30 minutes.

A colleague suggests increasing the warehouse size from Small to Large to fix the slow morning load.

What is actually happening, and what should the team do?

- ☐ A The warehouse is undersized. Increase it to Large as suggested
- ☐ B The warehouse is cold-starting. Switch to a serverless SQL warehouse
- ☐ C The Delta tables need OPTIMIZE, which is why the first query is slow
- ☐ D The result cache is empty each morning. Increase auto-stop to 12 hours

◆ PAUSE AND DECIDE

What is actually happening, and what should the team do?

A Increase the size to Large

A bigger warehouse still takes about four minutes to start. You would pay more for every hour of the day and the morning wait would not move.

C Run OPTIMIZE on the tables

File layout affects every query equally. It would not produce fast queries all afternoon and one slow query at nine.

B Switch to serverless

Correct. Four minutes is the documented cold-start for classic and pro warehouses. Serverless starts in seconds, which removes the wait entirely.

D Keep it running for 12 hours

It would work, by paying for an idle warehouse all night. It also misreads the cause: this is start-up time, not cache warming.

THE SIGNAL

'Four minutes, then fast all day' is a signature. It is the shape of a cold start, not of a sizing or layout problem.

A data science team needs to train models using Databricks Runtime ML on GPU instances. The workspace is governed by Unity Catalog, and the platform team requires that all data access is enforced through it.

Four people need to use the same compute resource. Any tables they create must be owned by the team rather than by one individual.

How should the compute be configured?

- ☐ A Standard access mode, which supports multiple users and Unity Catalog
- ☐ B No isolation shared access mode, so all four can use one resource
- ☐ C Dedicated access mode assigned to a group containing the four users
- ☐ D Four separate dedicated resources, one assigned to each user

◆ PAUSE AND DECIDE

How should the compute be configured?

A Standard access mode

Normally the right default, and that is the trap. Standard does not support Databricks Runtime ML or GPU workloads, so it cannot run this at all.

C Dedicated, assigned to a group

Correct. Dedicated supports Runtime ML and GPUs, group assignment lets all four share it, and objects created are owned by the group.

B No isolation shared

Unity Catalog permissions do not apply on this mode, which breaks the platform team's stated requirement outright.

D Four individual dedicated resources

It would run the workload, but four GPU clusters instead of one, and each person would own their own tables. Both requirements missed.

THE SIGNAL

Two constraints pull in opposite directions: Runtime ML forces dedicated, sharing suggests standard. Group-dedicated is what resolves them.

A regulatory report must be produced by 06:00 every day. The job takes about 90 minutes on a 12-worker cluster. To cut cost, an engineer configures the cluster to use spot instances for every node, including the driver.

Over the following fortnight the job fails three times. Each failure happens partway through, and the run has to be restarted from the beginning.

What should the engineer change?

- ☐ A Remove spot instances entirely and use on-demand for all nodes
- ☐ B Keep spot for the workers but move the driver to an on-demand instance
- ☐ C Increase the cluster from 12 workers to 24 so the job finishes before reclamation
- ☐ D Add an instance pool so reclaimed nodes are replaced immediately

◆ PAUSE AND DECIDE

What should the engineer change?

A On-demand for everything

It fixes the failures and throws away all the saving. The question asks what to change, and this changes more than the problem requires.

C Double the workers

Reclamation is not a race against the clock; it can happen at any moment. Twice the nodes is twice the exposure, not less.

B Spot workers, on-demand driver

Correct. Losing a spot worker costs a retry of its tasks. Losing the spot driver kills the run, which is exactly the failure described.

D Add an instance pool

Pools speed up start-up. They do not stop the cloud provider reclaiming a spot instance, and they do not protect the driver.

THE SIGNAL

Spot is safe where work can be retried and unsafe where it cannot. That maps exactly onto executors and the driver.

A team has forty small jobs that each run for about two minutes, triggered irregularly through the day. They currently run on classic jobs compute, which takes roughly four minutes to start each time.

The team's bill shows far more compute time than the work should require, and users complain that short jobs feel slow.

What is the best change?

- ☐ A Move the jobs to serverless compute
- ☐ B Move the jobs onto a shared all-purpose cluster that stays running all day
- ☐ C Increase the cluster size so each two-minute job finishes faster
- ☐ D Combine all forty jobs into one job that runs four times a day

◆ PAUSE AND DECIDE

What is the best change?

- A

 Move to serverless compute
Correct. Start-up drops from four minutes to seconds, so the overhead that dominates a two-minute job disappears, and there is no idle cost.
- B

 A shared all-purpose cluster all day
It removes start-up and replaces it with a cluster billing all day at the higher all-purpose rate. One form of waste traded for a worse one.
- C

 Increase the cluster size
The two minutes of work is not the problem; the four minutes of start-up is. A bigger cluster starts just as slowly and costs more.
- D

 Combine into one job run four times daily
It cuts start-ups from forty to four, but the jobs are triggered irregularly. Batching destroys the timing the triggers exist to provide.

THE SIGNAL

When start-up time exceeds run time, the overhead is the workload. Fix the overhead, not the work.

Compute questions almost always reduce to two things: **are you paying the right rate,** and **is anything billing while doing nothing.**